

Embedded Systems

Assignment 1: From FPGA (PL) to CPU (PS)

Adam Kolodziej

adkol23@student.sdu.dk

Contents

1. Introduction and hardware setup.....	2
2. Vivado block diagram – learning and testing.....	2
2.1. Receiving data via Programmable Logic (PL).....	2
2.2. BRAM via Processing System (PS).....	3
3. Entire solution.....	4

1. Introduction and hardware setup.

The purpose of this paper is to show how to communicate between FPGA and the CPU using BRAM and UART.

Data transferred from UART port via FTDI (USB to serial UART converter) is read by the FPGA circuit and stored in BRAM memory. Then data from BRAM is read by the CPU and transferred back to PC via USB port.

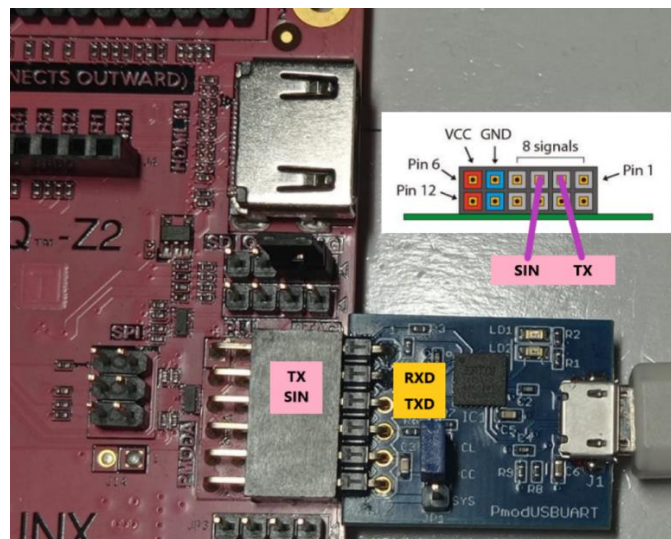


Figure 1 Hardware setup

2. Vivado block diagram – learning and testing.

2.1. Receiving data via Programmable Logic (PL).

It's performed using FTDI converter. Final solution needs only the wire connecting RX of the PmodA with FTDI board because UART is only used for receiving data from PC. For the testing purposes two wires are needed: RX – which receives data, and TX – which sends data. To check if received data is correct I use TX to send back to PC, so I can print it out into the terminal. I decided to test this part this way because it seems to be really reliable and at the same time simple to perform.

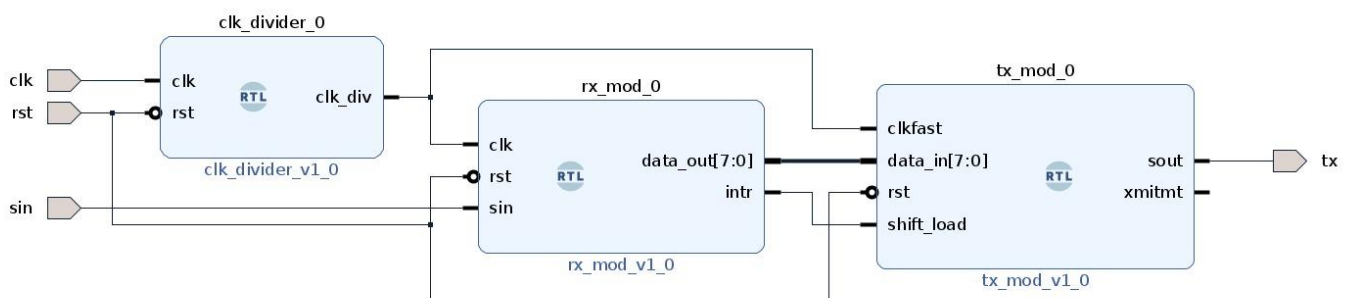


Figure 2 Test diagram of serial reading part

Video:

<https://youtu.be/C3RHizULnZQ>

This part of the program uses three signals: clock (clk), reset (rst) and data (sin). Baudrate is set to 115200 MHz so to be able to receive correct messages it is necessary to use clock divider to reduce system clock frequency.

By default RX block has bus of 10 bits of ones. When the starting bit appears, it starts reading 8 bits of data and the last stop bit. Then the “intr” output is set to high which indicates that message was received.

2.2. BRAM via Processing System (PS).

To understand how BRAM memory works I used simulation module. Diagram consists only of Block Memory Generator. I wanted to see what kind of data I need to write to inputs of this block to be able to save data on BRAM. I did it by forcing constants or clock variables (Figure 4). It is important to set enable pins to 1, reset to 0, correctly set up clock and in my case web to f because I was transferring data to B port. Then, to see how data is saved to the BRAM, I entered a few lines of code to Tcl Console:

```
add_force {/design_1_wrapper/dinb_0} -radix hex {0 0ns}
run 200 ns
add_force {/design_1_wrapper/dinb_0} -radix hex {15 0ns}
run 300 ns
add_force {/design_1_wrapper/dinb_0} -radix hex {ff 0ns}
run 200 ns
add_force {/design_1_wrapper/dinb_0} -radix hex {60 0ns}
run 500 ns
add_force {/design_1_wrapper/dinb_0} -radix hex {0 0ns}
run 500 ns
```

Hex data in “{}” is value and delay. After each force data, I run the program for a certain amount of time. In the video I show how data at 4th address changes.

Video:

<https://youtu.be/NpWxnGRp1Mg>

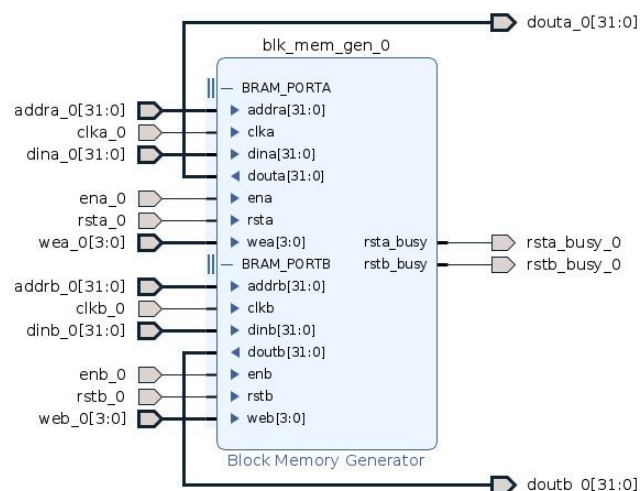


Figure 3 Block Memory Generator test

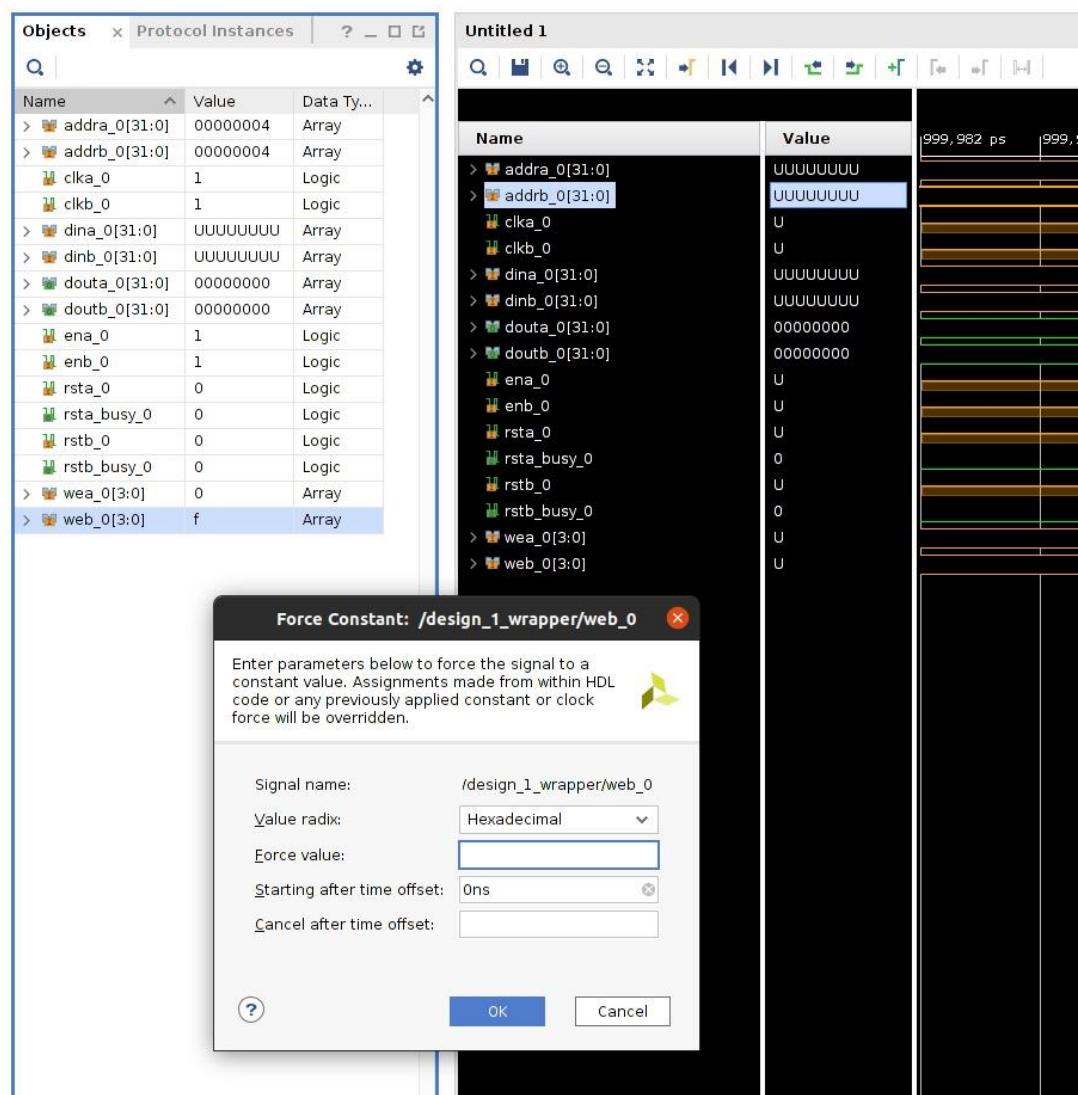


Figure 4 Force input variables of Block Memory Generator

3. Entire solution.

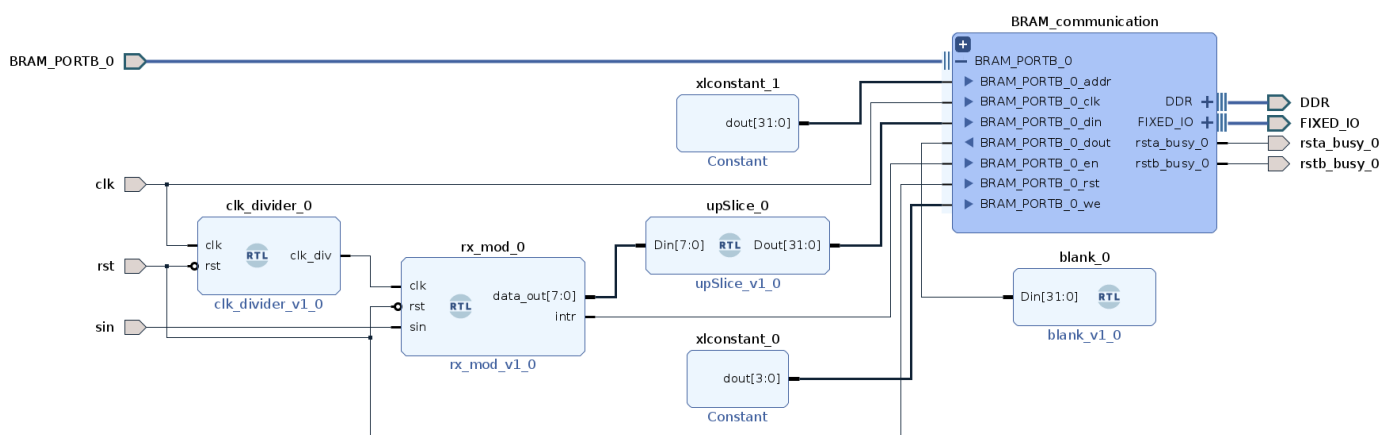


Figure 5 Entire solution

First part of the digital circuit stores a message coming from a PL UART. The way how it works is described in 2.1. As I mentioned before, `tx_mod_0` is not useful in the final solution. Instead of it, I had to create and add `upSlice` block. The data coming out of `rx_mod_0` is 8-bit. It can't be transferred directly to `BRAM_PORTB_0_din` because it can receive only 32-bit data. I solved this creating simple block which fills the rest of 32-bit data with "ones":

```

entity upSlice is
    Port ( Din : in STD_LOGIC_VECTOR (7 downto 0);
          Dout : out STD_LOGIC_VECTOR (31 downto 0));
end upSlice;

Architecture Behavioral of upSlice is

begin
    Dout(31 downto 8) <= (others => '1');
    Dout(7 downto 0) <= Din;

end Behavioral;

```

Figure 6 upSlice

I also had to create a few constants:

- xlconstant 1 – contains the BRAM address where the data will be stored – in my case it's address 1.
- xlconstant 0 – enables writing to PortB.

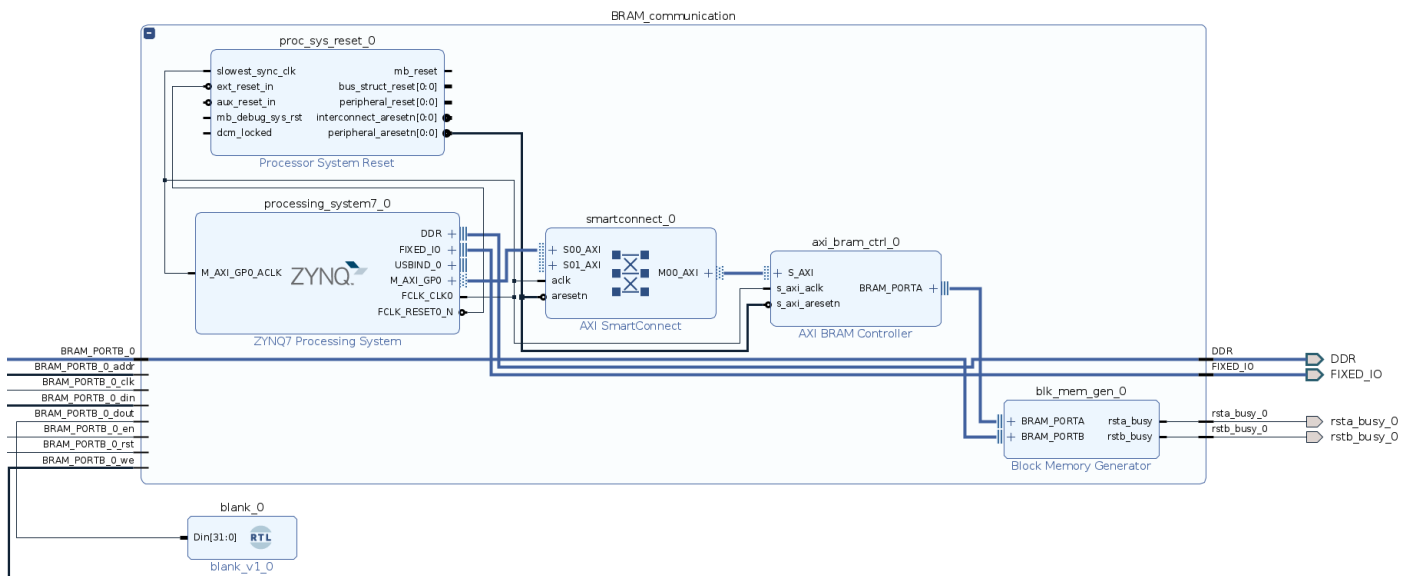


Figure 7 BRAM_communication hierarchy

BRAM communication hierarchy consists of blocks used to save data in BRAM:

Zynq Processing System – it's a block which is used to configure the PS so that the Zynq has booted correctly and support to peripherals is run. Basically it allows project to use the Zynq board.

It's connected with Processor System Reset which can reset entire processor system (interconnect and peripherals as well).

Communication with RAM takes place via AXI protocol. The Advanced eXtensible Interface is defined by ARM as part of Advanced Microcontroller Bus Architecture. AXI uses 5 channels to establish communication – 2 to read (address, data) and 3 to write (address, data, response).

First used AXI block is AXI SmartConnect which performs connection between master and slave devices. That's why it is used to transfer data between Zynq PS (master) and AXI BRAM Controller (slave).

AXI BRAM Controller is an AXI Endpoint slave IP which allows communication to local block RAM. It uses BRAM PORTA to communicate with Block Memory Generator tested at 2.2.

Meanwhile, BRAM PORT B is used to transfer my data (written by me at the terminal and received via UART). To do it, I used “`sudo screen /dev/ttyUSB<num> 115200`”.

Video:

<https://youtu.be/K6ijgwWpIWQ>

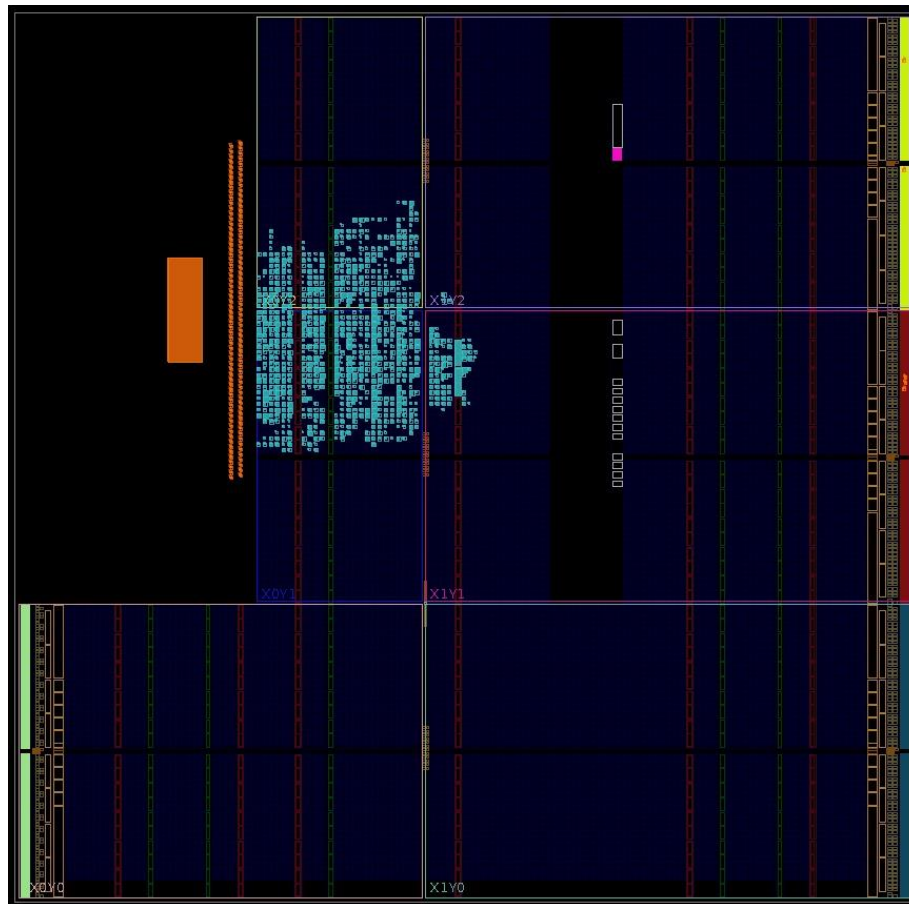


Figure 8 Implementation

```

27 int main()
28 {
29     init_platform();
30
31     initBRAM();
32
33     xil_printf("BRAM initialization complete.\r\n");
34
35     int addr_value = 1;
36
37     char data = 0;
38
39     xil_printf("Enter memory value for address %d:\r\n", addr_value);
40
41     while(1)
42     {
43         data = MYMEM_u(addr_value);
44
45         if(data != 0)
46         {
47             char spaceKeycode = 32;
48             switch (data)
49             {
50                 case 8: //keycode -> BACKSPACE
51                     xil_printf("\b"); //going back
52                     xil_printf("%c", spaceKeycode); //overwriting old char with spacebar
53                     xil_printf("\b"); //going back again
54                     break;
55                 case 13: //keycode -> ENTER
56                     xil_printf("\n\r"); //newline
57                     break;
58                 default:
59                     xil_printf("%c", data);
60                     break;
61             }
62         }
63
64         MYMEM_u(addr_value) = 0;
65         usleep(1000);
66     }
67
68     cleanup_platform();
69     return 0;
70 }

```

Figure 9 c code written in Vitis

C code is pretty simple. I read memory value from provided address (in my case 1). If there is something different than "0", it is checked if it's not the keycode matching BACKSPACE or ENTER. ENTER just moves to new line. BACKSPACE behavior was a bit more complicated. It's impossible to delete the char written before but it is possible to overwrite it. That's why I move cursor back, overwrite the char with spacebar and move cursor back again. If no special case is detected, char saved in the BRAM is simply written. After that memory value is set back to "0" and the process begins again.